

Laboratorio: Arreglos y Vectores

Del registro de notas al análisis de resultados

Laboratorio: Analizador de notas

Meta del laboratorio

En aproximadamente **30 minutos** desarrollarás un programa en C++ utilizando un **arreglo de 5 notas**.

Esta vez el programa **no será construido paso a paso por el docente**.

La meta es comprobar que ya puedes transformar por tu cuenta la idea:

casilla → índice → valor → recorrido → procesamiento

1. Situación

Un docente necesita un pequeño programa para registrar las notas de **5 estudiantes** y obtener información básica del grupo.

El programa deberá almacenar todas las notas en un **arreglo** y utilizar ciclos para recorrerlo.

Regla principal

No debes crear variables como:

```
nota1, nota2, nota3, nota4, nota5
```

Las cinco notas deben almacenarse en **un solo arreglo**.

2. Requerimiento

Construye un programa en C++ que permita:

1. Crear un arreglo para almacenar **5 notas**.
2. Solicitar al usuario las cinco notas.
3. Mostrar todas las notas almacenadas junto con su **índice**.
4. Calcular la **suma** de las notas.
5. Calcular el **promedio** del grupo.
6. Identificar la **nota mayor**.
7. Identificar la **nota menor**.
8. Contar cuántos estudiantes **aprobaron**.
9. Contar cuántos estudiantes **reprobaron**.

Considere que:

```
Aprobado    → nota >= 61  
Reprobado   → nota < 61
```

3. Ejemplo de funcionamiento

El resultado de tu programa podría verse aproximadamente así:

```
=====
          ANALIZADOR DE NOTAS
=====

Ingrese nota [0]: 75
Ingrese nota [1]: 60
Ingrese nota [2]: 90
Ingrese nota [3]: 45
Ingrese nota [4]: 80

=====
          NOTAS REGISTRADAS
=====

Indice 0 -> 75
Indice 1 -> 60
Indice 2 -> 90
Indice 3 -> 45
```

```
Indice 4 -> 80
```

```
=====
                        RESULTADOS
=====
```

```
Suma: 350
Promedio: 70
Nota mayor: 90
Nota menor: 45
Aprobados: 3
Reprobados: 2
```

i Importante

El ejemplo muestra **cómo debería comportarse el programa**, pero no indica cómo debes programarlo.
Esa parte te corresponde resolverla.

4. Antes de programar

Dedica unos minutos a pensar.

¿Qué necesitamos almacenar?

Completa mentalmente o en tu cuaderno:

Elemento	¿Variable o arreglo?	Tipo sugerido
5 notas		
suma		
promedio		
nota mayor		
nota menor		
cantidad de aprobados		
cantidad de reprobados		

¿Cuántos recorridos necesitamos?

Piensa:

- ¿Podemos ingresar las notas utilizando un `for`?
- ¿Podemos mostrar las notas con otro `for`?
- ¿Podemos calcular varios resultados durante un mismo recorrido?

 No comiences escribiendo todo el programa

Primero identifica:

Entradas → Procesos → Salidas

Después programa una parte, compila, prueba y continúa.

5. Plan de trabajo sugerido

Etapla 1 — Registrar las notas

Primero consigue que tu programa:

- declare el arreglo;
- recorra sus posiciones;
- solicite las cinco notas;
- almacene correctamente cada valor.

Cuando funcione, continúa.

Etapla 2 — Mostrar el arreglo

Recorre nuevamente las posiciones y muestra:

```
Indice 0 -> ...  
Indice 1 -> ...  
Indice 2 -> ...
```

Comprueba que los datos coincidan con lo que ingresaste.

Etapla 3 — Suma y promedio

Utiliza los valores almacenados para obtener:

- suma;
- promedio.

Etapla 4 — Mayor y menor

Ahora determina:

- la nota más alta;
- la nota más baja.

💡 Una pista, no la solución

Ya tienes cinco valores almacenados.

Pregúntate:

¿Con qué valor existente del arreglo podría comenzar la variable que guardará el mayor y el menor?

Evita inventar valores arbitrarios.

Etapla 5 — Aprobados y reprobados

Durante el recorrido analiza cada nota:

```
¿La nota es mayor o igual a 61?
```

Necesitarás llevar dos contadores.

6. Tiempo de desarrollo

Tiempo	Actividad
5 min	Analizar el problema
20 min	Programar y probar
5 min	Revisar resultados
30 min	Total

! Durante estos 30 minutos

Intenta resolver el problema utilizando lo aprendido en clase.

Antes de pedir ayuda:

1. lee el mensaje del compilador;
2. identifica en qué parte ocurre;
3. revisa índices, ciclos y condiciones;
4. prueba nuevamente.

7. Casos de prueba

Antes de considerar terminado el programa, realiza al menos estas pruebas.

Prueba 1

75
60
90
45
80

Comprueba manualmente los resultados.

Prueba 2

Utiliza cinco notas elegidas por ti.

Antes de ejecutar el programa, calcula en papel:

- suma;
- promedio;
- mayor;
- menor;
- aprobados;
- reprobados.

Luego compara tus resultados con los del programa.

8. Preparémonos para explicar

Al finalizar el tiempo de desarrollo, algunos estudiantes presentarán su solución.

No basta con que el programa funcione.

Debes poder explicar:

1. ¿Cómo declaraste el arreglo?
2. ¿Por qué los índices comienzan en 0?
3. ¿Cómo ingresaste las notas?
4. ¿Cómo recorriste el arreglo?
5. ¿Cómo calculaste la suma?
6. ¿Cómo encontraste la nota mayor?
7. ¿Cómo encontraste la nota menor?
8. ¿Cómo contaste aprobados y reprobados?

Regla de la exposición

El código puede compilar perfectamente y aun así existir algo que mejorar.

Durante las exposiciones compararemos **formas de pensar y resolver el problema**, no solamente resultados.

9. Reto adicional

Si terminaste antes de los 30 minutos, agrega una búsqueda.

El programa deberá solicitar una nota:

```
Ingrese la nota que desea buscar: 90
```

Luego deberá recorrer el arreglo.

Si la encuentra:

```
Nota encontrada en el indice 2
```

Si no existe:

```
La nota no se encuentra registrada.
```

 Pregunta

¿Y qué ocurriría si la misma nota aparece más de una vez?

¿Tu programa mostraría solamente la primera posición o todas las posiciones?

10. Alto aquí

 No continúes todavía

La siguiente sección contiene una posible solución.

Primero:

programa → **prueba** → **explica** → **compara**

Después de las exposiciones analizaremos juntos la solución.

Solución propuesta

Una posible solución es:

```
#include <iostream>
using namespace std;

int main() {

    const int TAM = 5;
    int notas[TAM];

    int suma = 0;
    double promedio;

    int aprobados = 0;
    int reprobados = 0;

    // Ingreso de las notas
    for (int i = 0; i < TAM; i++) {
        cout << "Ingrese nota [" << i << "]: ";
        cin >> notas[i];
    }

    // Inicializamos mayor y menor con un valor real del arreglo
    int mayor = notas[0];
    int menor = notas[0];

    // Procesamiento
    for (int i = 0; i < TAM; i++) {

        suma = suma + notas[i];

        if (notas[i] > mayor) {
            mayor = notas[i];
        }

        if (notas[i] < menor) {
            menor = notas[i];
        }

        if (notas[i] >= 61) {
```

```

        aprobados++;
    }
    else {
        reprobados++;
    }
}

promedio = suma / static_cast<double>(TAM);

// Mostrar las notas
cout << "\n===== " << endl;
cout << "          NOTAS REGISTRADAS" << endl;
cout << "===== " << endl;

for (int i = 0; i < TAM; i++) {
    cout << "Indice " << i
        << " -> " << notas[i] << endl;
}

// Resultados
cout << "\n===== " << endl;
cout << "          RESULTADOS" << endl;
cout << "===== " << endl;

cout << "Suma: " << suma << endl;
cout << "Promedio: " << promedio << endl;
cout << "Nota mayor: " << mayor << endl;
cout << "Nota menor: " << menor << endl;
cout << "Aprobados: " << aprobados << endl;
cout << "Reprobados: " << reprobados << endl;

return 0;
}

```

11. Analicemos la solución

El arreglo

```

const int TAM = 5;
int notas[TAM];

```

Tenemos cinco casillas:

```
notas[0]
notas[1]
notas[2]
notas[3]
notas[4]
```

El ingreso

```
for (int i = 0; i < TAM; i++) {
    cin >> notas[i];
}
```

El mismo cin trabaja cada vez con una posición diferente.

Mayor y menor

```
int mayor = notas[0];
int menor = notas[0];
```

En lugar de inventar un valor inicial, utilizamos un dato que sabemos que pertenece al arreglo.

Después comparamos:

```
if (notas[i] > mayor) {
    mayor = notas[i];
}
```

y:

```
if (notas[i] < menor) {
    menor = notas[i];
}
```

Aprobados y reprobados

```
if (notas[i] >= 61) {  
    aprobados++;  
}  
else {  
    reprobados++;  
}
```

Aquí combinamos tres conocimientos:
arreglos + ciclos + decisiones.

12. Pregunta final

Observa todo el programa.

💡 ¿Qué cambió realmente?

Antes necesitábamos pensar en:

```
nota1  
nota2  
nota3  
nota4  
nota5
```

Ahora pensamos en:

```
notas[i]
```

La variable *i* nos permite trabajar con **cualquier posición**.
Esa es una de las ideas más importantes de los arreglos.

13. Cierre del laboratorio

Hoy comprobamos que un arreglo no sirve únicamente para guardar muchos datos.
Al combinar:

```
Arreglo
+
Ciclo
+
Condiciones
```

podemos convertir un conjunto de datos en **información**:

- promedio;
- máximo;
- mínimo;
- conteos;
- búsquedas.

Misión cumplida

Si pudiste construir y explicar este programa, ya no estás simplemente utilizando arreglos.

Estás comenzando a **procesar colecciones de datos mediante algoritmos**.